

Universidad Torcuato Di Tella

Métodos Analíticos y Programación

Prof. Hernán Czemerinski

Simulación Montecarlo (II)

Repaso

Cuando obtener resultados exactos es complicado, la **simulación Montecarlo** permite estimarlos a partir de experimentos aleatorios en contextos con **incertidumbre** o **variabilidad**.

- ▶ Repetimos muchas veces un experimento aleatorio.
- ▶ Registramos los resultados obtenidos.
- ▶ Calculamos promedios, probabilidades u otras medidas de interés.

Idea clave: **simular muchas veces** y observar el comportamiento del sistema.

Repaso

Cuando obtener resultados exactos es complicado, la **simulación Montecarlo** permite estimarlos a partir de experimentos aleatorios en contextos con **incertidumbre** o **variabilidad**.

Ejemplo: estimar la probabilidad de que la suma de dos dados sea mayor que 8 al lanzarlos.

```
import random

N = 10000
 exitos = 0

for i in range(N):
    dado1 = random.randint(1,6)
    dado2 = random.randint(1,6)
    if dado1 + dado2 > 8:
        exitos = exitos + 1

prob = exitos / N
print('Probabilidad estimada:', prob)
```

Cuanto más repeticiones hagamos, más precisa será la estimación.

Experimentos con duración variable

En algunos casos, **el experimento** no tiene una cantidad fija de pasos, sino que **termina cuando se cumple cierta condición**.

Ejemplo: estimar cuántos lanzamientos de un dado, en promedio, se necesitan para obtener un 6.

```
import random

N = 10000
total_tiradas = 0

for i in range(N):
    tiradas = 0
    while random.randint(1,6)!=6:
        tiradas = tiradas + 1
    total_tiradas = total_tiradas + tiradas

promedio = total_tiradas / N
print(promedio)
```

Experimentos con duración variable

En algunos casos, **el experimento** no tiene una cantidad fija de pasos, sino que **termina cuando se cumple cierta condición**.

Ejemplo: estimar cuántos lanzamientos de un dado, en promedio, se necesitan para obtener un 6.

Solución equivalente (¡y mejor programada!):

```
import random

def correr_experimento():
    ''' Devuelve cuántos tiros fueron necesarios antes de obtener un 6. '''
    tiradas = 0
    while random.randint(1,6)!=6:
        tiradas = tiradas + 1
    return tiradas

# Cuerpo principal del programa
N = 10000
total_tiradas = 0
for i in range(N):
    total_tiradas = total_tiradas + correr_experimento()
promedio = total_tiradas / N
print(promedio)
```

Experimentos con duración variable

Ejercicio 1.

En un centro de atención telefónica, recibimos llamadas de clientes con distintos motivos: consultas sobre productos, reclamos, soporte técnico y facturación. Nos interesa analizar cuántas llamadas, en promedio, deben atenderse hasta recibir la primera **consulta sobre facturación**.

Usando simulación Montecarlo, podemos estimar este valor sin necesidad de depender de datos históricos, simplemente simulando llamadas aleatorias y contabilizando cuándo ocurre la primera consulta de facturación.

Se pide escribir un programa en Python que:

1. Simule cada llamada de cliente usando `random.random()`, considerando que la probabilidad de que la llamada sea sobre facturación es 0,2.
2. Repita la simulación **10.000 veces**.
3. Calcule el promedio de llamadas necesarias hasta recibir la primera consulta sobre facturación.
4. Muestre el resultado por consola.

Pista: el experimento termina cuando se obtiene la primera consulta de facturación y cada llamada es independiente de las anteriores.

Experimentos con memoria

En ocasiones, la finalización no depende únicamente del resultado actual, sino de los resultados previos. Es decir, el experimento tiene **memoria**. En estos casos, para determinar si el experimento termina, debemos **guardar información de eventos anteriores**.

Ejemplo: estimar cuántos lanzamientos de una moneda se necesitan, en promedio, hasta obtener **dos caras consecutivas**.

```
def correr_experimento():
    ''' Devuelve la cantidad de lanzamientos para dos caras consecutivas. '''
    tiradas = 0
    caras_consecutivas = 0
    while caras_consecutivas < 2:
        lanzamiento = random.choice(['cara', 'ceca'])
        tiradas = tiradas + 1
        if lanzamiento == 'cara':
            caras_consecutivas = caras_consecutivas + 1
        else:
            caras_consecutivas = 0
    return tiradas

# Cuerpo principal del programa
N = 10000
total_tiradas = 0
for i in range(N):
    total_tiradas = total_tiradas + correr_experimento()
promedio = total_tiradas / N
print(promedio)
```

Experimentos con memoria

Ejercicio 2.

Un sitio web de comercio electrónico analiza el comportamiento de sus usuarios. Nos interesa estimar cuántas visitas realiza, en promedio, un usuario hasta que ocurre el primer caso de **dos compras consecutivas**.

Sabemos que en cada visita, la probabilidad de que un usuario realice una compra es 0,3, y que cada visita es independiente de las anteriores.

Se pide escribir un programa en Python que:

1. Simule el comportamiento de un usuario: en cada visita, genera un número aleatorio con `random.random()` y considera que hay compra si el valor es menor que 0,3.
2. El experimento termina cuando se registran dos compras consecutivas.
3. Repita la simulación **10.000 veces**.
4. Calcule el promedio de visitas necesarias hasta que ocurran dos compras seguidas.
5. Muestre el resultado por consola.

El resultado exacto es 14.44. ¿Qué tan cerca estuvo la estimación obtenida por simulación?

Simulación Montecarlo y decisiones bajo riesgo

Hasta ahora usamos simulación Montecarlo para estudiar fenómenos aleatorios y estimar valores esperados. Pero también es una herramienta fundamental en la **toma de decisiones bajo incertidumbre**.

- ▶ En muchos contextos reales (finanzas, operaciones, planificación, ingeniería), las decisiones deben tomarse sin conocer con certeza los resultados futuros.
- ▶ La simulación Montecarlo permite **modelar escenarios posibles** y estimar **riesgos y rendimientos esperados**.
- ▶ En lugar de buscar una única respuesta ?correcta?, exploramos cómo varían los resultados ante distintas condiciones aleatorias.

Ejemplo: comparar inversiones con distinto nivel de riesgo mediante simulación de resultados anuales.

Esto da origen al uso de Montecarlo en análisis financiero, seguros y evaluación de proyectos, donde el riesgo y la incertidumbre son componentes centrales de la decisión.

Decisiones bajo riesgo

Ejercicio 3.

Un inversor puede elegir entre dos alternativas de inversión:

- ▶ un **bono seguro**, que rinde 2 % anual,
- ▶ un **proyecto riesgoso**, que tiene:
 - ▶ 15 % de probabilidad de pérdida del 20 %,
 - ▶ 85 % de probabilidad de ganancia del 5 %.

Se pide escribir un programa en Python que:

1. Simule **100.000 años**, generando resultados aleatorios para cada opción.
2. Estime el rendimiento promedio del bono y del proyecto riesgoso.
3. Compare los resultados y discuta cuál conviene elegir según el rendimiento esperado.

Sugerencia: usar `random.random()` para decidir si el proyecto riesgoso gana o pierde en cada simulación.

Este tipo de simulaciones permite analizar decisiones financieras, de seguros o de inversión bajo incertidumbre.

Repaso de la clase

Montecarlo: **repetir experimentos aleatorios** y analizar resultados.

Vimos:

- ▶ Experimentos con duración variable
- ▶ Experimentos con memoria
- ▶ Decisiones bajo riesgo

Con lo visto, ya pueden completar la sección 3 de la guía de ejercicios.